



updated 11 August 2017

Introductions

Who is this Guy ?

Proprietary/Principal Trading

Brian Peterson:

- quant, author, open source advocate
- [DV Trading](#) Partner | Head Trader | Automated Trading Strategies
- Senior Lecturer (Part-Time), University of Washington [Computational Finance and Risk Management](#)
- author or co-author of over 10 packages for using R in Finance
- organization admin for [R](#)'s participation in [Google Summer of Code](#)

Proprietary Trading:

- proprietary or principal traders are a specific "member" structure with the exchanges
- high barriers to entry, large up-front capital requirements
- many strategies pursued in this structure have capacity constraints
- benefits on the other side are low fees, potentially high leverage
- money management needs to be very focused on drawdowns, leverage, volatility

So what is Machine Learning?

- after Artificial Intelligence models failed to produce human-like intelligence
- Machine Learning was adopted as the term to include a huge range of (mostly) non-parametric computational inference models
- the goal is to provide inference from data

see:

- Domingos (2015) *The Master Algorithm*

- Efron & Hastie (2016) *Computer Age Statistical Inference* for book-length treatments.

Data and Features

So it's all about the data?

- market data capture going from ~100GB/day to ~1+TB/day as more information sent by exchanges
- many vendors, including huge players like Reuters and IBM, are providing additional parsed data sources
- top users of ML in trading are collecting, curating, and cultivating rare or private data sources
- now common to have tens to hundreds of additional descriptive time series in addition to market data



<https://xkcd.com/1838/>¹

What about the Features?

- how do we go from descriptive to predictive?
- the raw data is usually not the main feature in time series prediction
- feature selection (Guyon and Elisseeff 2003) ranks and filters multiple possible features
- it is often better to first engage in pro-active feature engineering:

- transformation of data to regularize scale, variation
- coding of data into categorical or integer states
- adding features that are indicative of the objective
- using extra models for things like seasonality or colinearity

About the Models

Rules Engines

- rules created by experts have largely fallen out of favor (*or have they?*)
- knowledge engineering and expert systems enjoyed a brief ascendancy in the 1980's
- notably before computers were fast enough to employ statistical approaches
- many of the most successful users of ML spend significant time and money crafting features from data
- perhaps knowledge encoded as features is the most important legacy of the expert systems
- Decision Tree R Packages: [rpart](#) [party](#) [randomForest](#)

Optimization

- almost all machine learning has its roots in optimization
- many algorithms use optimization solvers or parts of optimization solvers
- optimization is also valuable in this context because it is one of the most common ML use cases
- for overviews of (primarily) convex optimization see the Simon's Institute talks by Ben Recht (2013a, (2013b))
- global stochastic models can handle non-smooth objectives, e.g.
 - Particle Swarms (Kennedy 2011), e.g. [psa](#)
 - Simulated Annealing (Kirkpatrick et al. 1983), e.g. [genSA](#)
 - Genetic Algos like Differential Evolution (Price, Storn, and Lampinen 2006, Zhang and Sanderson (2009)), e.g. [DEoptim](#)

... and *learn* or adapt as they gather more data

Gradient Descent

- gradient descent is the preference to move towards a local minima
- used widely in solving optimization problems
- the famous Perceptron (Rosenblatt 1958) is a gradient descent algorithm (and also the original neural network)
- information on Momentum for gradient descent in Goh (2017)
- [policy gradients](#) are formed when you add parameters to gradient descent, and update optimal parameters as more data is examined
- policy gradients are the key mechanism utilized to construct reinforcement learning agent based models
- see packages [ReinforcementLearning](#) and [MDPtoolbox](#)

Back Propagation

- backpropagation is the adjustment of previously established views or connections based on how new data differs from expectations
- related to policy gradients and gradient descent,
- the most famous models employing backpropagation are neural networks
- 'deep learning' is a neural network with more layers, requiring bigger computers and more data
- one of the oldest true 'learning' approaches as the model adapts itself (Chauvin and Rumelhart 1995, Horikawa, Furuhashi, and Uchikawa (1992))
- host of different neural network flavors, implemented in [neuralnet](#), [keras](#), [tensorflow](#)

Evolution

Crossover & Mutation

- all stochastic optimizers are to some degree population based
- evolutionary algorithms adopt this language from biology
- features critical to 'learning' optimizers are crossover and mutation
- crossover is the mixing of features from more successful trials (Spears and others 1992, Spears et al. (1993))
- mutation randomizes features (even if successful) to avoid local minima
- look at global solvers such as [DEoptim](#) or [pso](#)

Probabalistic Inference

- relationships between variables may often be known only with a certain probability
- state transitions also occur with a certain observable probability even if the underlying causes are unknown
- inference from these probabilities is the core of Monte Carlo, Bayesian methods (Cooper 1990, Neal (1993))
- inherently computationally intensive in ways that some earlier learning approaches were not
- see [Stan](#) (and [rstan](#))

Similarity

- similarity between training set and sample used to draw inferences
- applicable to both Euclidian and non-Euclidian feature spaces
- after pairwise similarity methodology is set, no need for access to the features themselves
- applied to regression, classification, and ranking (Chen et al. 2009)
- [k-means clusters](#), [SVM](#) models, [kernel](#) models all used to build similarity models

Getting It Wrong

Evaluating Overfitting in Machine Learning

Classical measures:

- specialized t -tests for ML algorithms
- cross-validation (but this makes the t -test challenging)
- p -value (but subject to p -hacking (Head et al. 2015))
- multiple-hypothesis testing (beware of false discovery rate (Efron and Hastie 2016, Ch. 15))
- mean squared forecast error (MSFE, RMSE)
- resampled errors (bootstrap, permutation tests, etc.)

ML-specific measures (Japkowicz and Shah 2011):

- Confusion Matrix
- Receiver Operating Characteristic (ROC) plots/stats or the related Cost Curves

Detecting Overfitting in Backtests

Multiple tests focused on overfitting of backtests also exist:

- **Walk Forward Analysis** : Pardo (2008) in [quantstrat](#)
- **Equity Curve Monte Carlo** : Tomasini and Jaekle (2009) in [blotter](#)
- **Round Turn Trade Monte Carlo** : Peterson (2017) in [quantstrat](#)
- **White's Reality Check** : from White (2000) and Hansen (2005) in [ttrTests](#)
- **k-fold cross validation** : randomly divide the sample of size $\backslash(T\backslash)$ into sequential sub-samples of size $\backslash(T/k\backslash)$. (Hastie, Tibshirani, and Friedman 2009)
- **CSCV sampling** (combinatorially symmetric cross validation): "generate $\backslash(S/2\backslash)$ testing sets of size $\backslash(T/2\backslash)$ by recombining the $\backslash(S\backslash)$ slices of the overall sample of size $\backslash(T\backslash)$ ". (Bailey et al. 2014, 17) in [pbo](#)
- **Multiple Hypothesis Testing** looks at Type I vs Type II error in evaluating backtests and at appropriate haircuts based on these probabilities. (Harvey and Liu 2015) in [quantstrat](#)

Using Machine Learning in Trading

A big computer, a complex algorithm and a long time does not equal science. - Robert Gentleman

Regression

- regression results may be the most common uses of machine learning
- you want the algorithm to estimate the value (mean/median), slope, etc. of some variable
- you may do this directly (e.g. [logistic regression](#), Bayesian regression via [Stan](#), [Random Forest](#), [Boost](#))
- or indirectly (MCMC, gradient descent, [state space model](#), optimization model with the correct objective)
- in many cases, simpler regression models (e.g. [dlm](#), [ridge regression](#), [LASSO](#)) may do just as well

Optimization

- not just a model feature, optimization is often the end goal
- consider whether simpler linear/quadratic/gradient optimizers will work – less model risk
- optimizer parameters for global optimizers can have huge impacts on model outputs and computation
- many problems may be recast as optimization problems:
 - capital allocation, see [PortfolioAnalytics](#)
 - risk management, PortfolioAnalytics and [PerformanceAnalytics](#) and [Meucci](#)
 - adaptive parameters
 - (optimal) process control, see [ReinforcementLearning](#) and [MDPtoolbox](#)

Classification / Clustering

- separation of observations into groups or states is classification or clustering (Japkowicz and Shah 2011)
- many ML models are used as classifiers
- proceeding from simple models is usually advisable
- key goal is typically to *label* the observed data or state
- so that some other model can take action based on that label/state
- common classification uses:
 - volatility regimes, e.g. [Markov Switching GARCH](#)
 - country/sector/volatility clustering
 - bull/bear market classification
 - trending/consolidating market

Composite Models

- lots of little models
- ensembles, boosting, weak learners
- in trading, this means mostly feature engineering
- or combining weak alpha trading models
- Worldquant (Tulchinsky 2015), Kepos (Carhart/Litterman), Jump, Renaissance, Bridgewater, Citadel, others all doing this to one degree or another

Doing this well takes scale!

Model Discovery

- the 'Holy Grail' of ML in trading

Like the Grail, mostly a myth

- feature engineering is what makes models work
- many good features and data sets make model 'discovery' possible
- focus on curating your data, building features
- with enough data, models, and features, you have broad choice of algorithms

Questions

Thanks

Thank You for Your Attention

Thanks to my team, and my family, who make it possible.

©2017 Brian G. Peterson brian@braverock.com



This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/)

Code to apply the techniques discussed here may be found in the *mlr*, *TensorFlow*, *keras*, *blotter*, *quantstrat*, *PerformanceAnalytics*, *PortfolioAnalytics*, and other **R** packages.

All views expressed in this presentation are those of Brian Peterson, and do not necessarily reflect the opinions or policies of DV Trading.

All remaining errors or omissions should be attributed to the author.

Installing quantstrat for backtesting in R

Development for these packages is on github, and

Install [devtools](#)

If on Windows, Install [Rtools](#)

Install *PerformanceAnalytics* and *FinancialInstrument* using *install.packages*

```
install.packages('FinancialInstrument')
```

```
install.packages('PerformanceAnalytics')
```

Install [blotter](#) and [quantstrat](#) as well as the current versions of [xts](#) and [quantmod](#) from github:

```
require(devtools)
```

```
install_github('joshuaulrich/xts')
```

```
install_github('joshuaulrich/quantmod')
```

```
install_github('braverock/blotter')
```

```
install_github('braverock/quantstrat')
```

Resources

Bailey, David H, Jonathan M Borwein, Marcos López de Prado, and Qiji Jim Zhu. 2014. "The Probability of Backtest Overfitting."

http://papers.ssrn.com/sol3/papers.cfm?abstract_id=2326253.

Chauvin, Yves, and David E Rumelhart. 1995. *Backpropagation: Theory, Architectures, and Applications*. Psychology Press.

Chen, Yihua, Eric K Garcia, Maya R Gupta, Ali Rahimi, and Luca Cazzanti. 2009. "Similarity-Based Classification: Concepts and Algorithms." *Journal of Machine Learning Research* 10 (Mar): 747–76.

Cooper, Gregory F. 1990. "The Computational Complexity of Probabilistic Inference Using Bayesian Belief Networks." *Artificial Intelligence* 42 (2-3). Elsevier: 393–405.

Domingos, Pedro. 2015. *The Master Algorithm: How the Quest for the Ultimate Lelearn Machine Will Remake Our World*. Penguin Books.

Efron, Bradley, and Trevor Hastie. 2016. *Computer Age Statistical Inference*. Vol. 5. Cambridge University Press.

Goh, Gabriel. 2017. "Why Momentum Really Works." *Distill*. doi:[10.23915/distill.00006](https://doi.org/10.23915/distill.00006).

Guyon, Isabelle, and André Elisseeff. 2003. "An Introduction to Variable and Feature Selection." *Journal of Machine Learning Research* 3 (Mar): 1157–82.
<http://www.jmlr.org/papers/volume3/guyon03a/guyon03a.pdf>.

Hansen, Peter R. 2005. "A Test for Superior Predictive Ability." *Journal of Business and Economic Statistics*.

Harvey, Campbell R., and Yan Liu. 2015. "Backtesting." SSRN.
<http://ssrn.com/abstract=2345489>.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Second Edition*. Springer.

Head, Megan L, Luke Holman, Rob Lanfear, Andrew T Kahn, and Michael D Jennions. 2015. "The Extent and Consequences of P-Hacking in Science." *PLoS Biol* 13 (3). Public Library of Science: e1002106.

Horikawa, S-I, Takeshi Furuhashi, and Yoshiki Uchikawa. 1992. "On Fuzzy Modeling Using Fuzzy Neural Networks with the Back-Propagation Algorithm." *IEEE Transactions on Neural Networks* 3 (5). IEEE: 801–6.

Japkowicz, Nathalie, and Mohak Shah. 2011. *Evaluating Learning Algorithms: A Classification Perspective*. Cambridge University Press.

Kennedy, James. 2011. "Particle Swarm Optimization." In *Encyclopedia of Machine Learning*, 760–66. Springer.
<http://www.cs.tufts.edu/comp/150GA/homeworks/hw3/reading6%201995%20particle%20swarming.pdf>.

Kirkpatrick, Scott, C Daniel Gelatt, Mario P Vecchi, and others. 1983. "Optimization by Simulated Annealing." *Science* 220 (4598). Washington: 671–80.

Neal, Radford M. 1993. "Probabilistic Inference Using Markov Chain Monte Carlo Methods." Department of Computer Science, University of Toronto Toronto, Ontario, Canada.

Pardo, Robert. 2008. *The Evaluation and Optimization of Trading Strategies*. Second ed. John Wiley & Sons.

Peterson, Brian G. 2017. "Developing & Backtesting Systematic Trading Strategies." DV Trading. <http://goo.gl/na4u5d>.

Price, Kenneth, Rainer M Storn, and Jouni A Lampinen. 2006. *Differential Evolution: A Practical Approach to Global Optimization*. Springer Science & Business Media.

Recht, Ben. 2013a. "Optimization I." <https://simons.berkeley.edu/talks/ben-recht-2013-09-04>.
———. 2013b. "Optimization II." <https://simons.berkeley.edu/talks/ben-recht-2013-09-04>.

Rosenblatt, Frank. 1958. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain." *Psychological Review* 65 (6). American Psychological Association: 386.

Spears, William M, and others. 1992. "Crossover or Mutation." *Foundations of Genetic Algorithms* 2: 221–37.

Spears, William, Kenneth De Jong, Thomas Bäck, David Fogel, and Hugo De Garis. 1993. "An Overview of Evolutionary Computation." In *Machine Learning: ECML-93*, 442–59. Springer.

Tomasini, Emilio, and Urban Jaekle. 2009. *Trading Systems: A New Approach to System Development and Portfolio Optimisation*.

Tulchinsky, Igor. 2015. *Finding Alphas: A Quantitative Approach to Building Trading Strategies*. John Wiley & Sons.

White, Halbert L. 2000. "System and Method for Testing Prediction Models and/or Entities." Google Patents. <http://www.google.com/patents/US6088676>.

Zhang, Jingqiao, and Arthur C Sanderson. 2009. *Adaptive Differential Evolution*. Springer.